Nested and chained conditions

An `if` inside an `if`, the same decision as one `elif` chain, then leap years and rock-paper-scissors.

THE QUESTION

How do you write a decision that depends on two or more questions?

THE INTUITION

Put an `if` inside another `if`'s block, or list the cases as one `if`/`elif` chain. Both draw the same decision tree.

KEY TAKEAWAY

Draw the tree, test one input per leaf plus the edge values, and line each `else` up with the `if` it answers.

two questions → nest: an if inside an if → or chain: if, elif, else → same decision tree → one test input per leaf

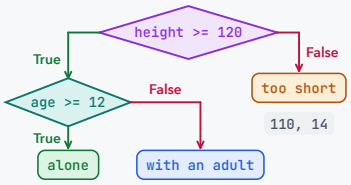
1 An `if` can sit inside another `if`

nested ifs

```
if height >= 120:
    if age >= 12:
        print("alone")
    else:
        print("with an adult")
else:
    print("too short")
```

level 1: 4 spaces level 2: 8 spaces

chips: the (height, age) runs that ended there



110, 14

130, 14 120, 12 130, 8

A **nested** `if` is an `if` inside another `if`'s block, asked only when the outer test is True: age matters only for riders 120 cm or taller. Each level adds 4 spaces. It is a **decision tree**: each diamond a test, each **leaf** an outcome. Test one input per leaf, plus the edges 120 and 12.

2 The same choice as one flat `elif` chain

one elif chain

```
if height < 120:
    print("too short")
elif age >= 12:
    print("alone")
else:
    print("with an adult")
```

first test flipped: the short branch (too short) goes first

height	age	nested	chain
130	14	alone	alone
120	12	alone	alone
130	8	with an adult	with an adult
110	14	too short	too short
120	11	with an adult	with an adult

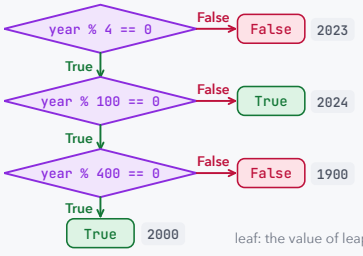
same output for all 16 pairs of heights 110, 119, 120, 130 and ages 8, 11, 12, 14

A **chain** lists the cases at one level and runs the first True branch (see the conditionals sheet). Once `height < 120` has taken the short riders, `elif age >= 12` sees only tall ones. Writing `height >= 120 and age >= 12` also works, but the next branch must test the height again. Nest when one question only makes sense after another.

3 The leap-year rule is three nested questions

nested, then print(leap)

```
if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
            leap = True
        else:
            leap = False
    else:
        leap = True
else:
    leap = False
```



leaf: the value of leap

A **leap year** divides by 4, except that a year dividing by 100 must also divide by 400. `year % 4 == 0` asks "divides exactly?". Each exception sits one level deeper, in the tree and in the code. The chips show where each year ended, recorded while the code ran.

4 Three ways to write it, and one wrong order

elif chain, strictest test first

```
if year % 400 == 0:
    leap = True
elif year % 100 == 0:
    leap = False
elif year % 4 == 0:
    leap = True
else:
    leap = False
```

one boolean expression

```
leap = ((year % 4 == 0 and
         year % 100 != 0)
        or year % 400 == 0)
```

all three match calendar.isleap for every year from 1 to 2400

	1900	2000	2024	2023
nested	False	True	True	False
expression	False	True	True	False
elif chain	False	True	True	False
% 4 first	True	True	True	False

One **Boolean expression** (a single True or False) with `and` and `or` also does it; brackets let it span lines. The elif chain asks the strictest case first: 400, then 100, then 4. Ask `% 4` first and 1900 is caught early as a leap year: that order gets 18 of the years 1 to 2400 wrong.

5 Rock-paper-scissors: nine cases in one chain

rps-1.py with line 1 set to u = ... and line 2 to c = ... (u: user, c: computer)

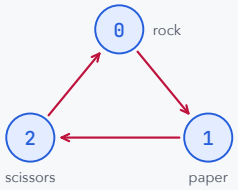
	c = "r"	c = "p"	c = "s"
u = "r" rock	Tie	c wins comp: p user: r	u wins comp: s user: r
u = "p" paper	u wins comp: r user: p	Tie	c wins comp: s user: p
u = "s" scissors	c wins comp: r user: s	u wins comp: p user: s	tie

user wins computer wins tie bug: "tie" here, "Tie" for the other two ties

as saved the file has u = "c", c = "p": prints nothing (no branch matches)

Rock beats scissors, scissors beat paper, paper beats rock. The course's `rps-1.py` has 9 branches of two comparisons joined by `and`. It names the right winner for all nine pairs but spells the ties two ways. As saved it sets `u = "c"`, not a move: a chain with no `else` does nothing when no test is True.

6 Three shorter versions of the same game



arrow: is beaten by. The move that beats u is `(u + 1) % 3`

the notebook's version with numbers

```
u = 0
c = 1
if u==c:
    print('tie')
elif (u+1)%3 == c:
    print("Comp wins")
else:
    print("User wins")
```

prints `Comp wins`

Ties first, nested, and numbers: each picks the same winner as rps-1.py for all 9 pairs.

Testing `u == c` first handles all three ties at once (7 branches). The nested version asks about `u`, then `c`, and ends in `else`, which also catches bad input: `u = "x"` prints `c wins comp: r user: x`. With the moves as 0, 1, 2, the computer wins when `(u + 1) % 3 == c`.

Simpler tests, and where each else belongs

Values that count as true or false, shorter conditions, De Morgan's laws, and matching `else` to `if`.

7 Any value can be a test: truthiness

falsy: bool(v) is False

`bool(False)`	`False`
`bool(0)`	`False`
`bool(0.0)`	`False`
`bool("")`	`False`
`bool([])`	`False`
`bool(None)`	`False`

truthy: bool(v) is True

`bool(True)`	`True`
`bool(5)`	`True`
`bool(-1)`	`True`
`bool(0.1)`	`True`
`bool("0")`	`True`
`bool(" ")`	`True`
`bool("False")`	`True`
`bool([0])`	`True`

```
name = ""
if name:
    print("Hello", name)
else:
    print("no name given")
```

```
name = ""
no name given
name = "Alice"
Hello Alice
```

An `if` accepts any value. Python asks `bool(value)`: zero, the empty string `""`, an empty **list** `[]` (lists come later) and `None` (Python's "no value") are **falsy**. Nearly everything else is **truthy**, even `"0"`, `" "` and `"False"`. So `if name:` means "if name is not empty".

8 Shorter conditions that say the same thing

before

```
if n % 2 == 0:
    even = True
else:
    even = False
```

after

```
even = n % 2 == 0
```

same even for n = -3 to 3

```
if x > 5:
    if x < 10:
        print("mid")
```

```
if 5 < x < 10:
    print("mid")
```

same output for x = 0 to 15

For a bool, `if done == True:` is just `if done:`. For other values they differ: `5 == True` is `False`, though 5 is truthy. A comparison is already True or False, so store it (in a function, `return n % 2 == 0`; see the functions sheet). Nested ifs with no `else` join with `and`.

9 De Morgan's laws: moving a `not` inside

a	b	not (a and b)	not a or not b	not (a or b)	not a and not b
False	False	True	True	True	True
False	True	True	True	False	False
True	False	True	True	False	False
True	True	False	False	False	False

Columns with the same colour match on every row, so each pair is the same test.

`not (13 <= age <= 19)` is the same as `age < 13 or age > 19`

checked for ages 0 to 30; at 12, 13, 19, 20 both give True, False, False, True

To push `not` inside brackets, negate each part and swap `and` with `or`: `not (a and b)` is `not a or not b`; `not (a or b)` is `not a and not b`. The table checks all four cases. A negated comparison flips, so "not a teenager" is `age < 13 or age > 19`.

10 An `else` pairs with the `if` in its own column

A: else under the inner if

```
1 if height >= 120:
2     if age >= 12:
3         print("alone")
4     else:
5         print("with an adult")
```

B: else under the outer if

```
1 if height >= 120:
2     if age >= 12:
3         print("alone")
4 else:
5     print("with an adult")
```

height, age	A prints	B prints
130, 8	with an adult	*(nothing)*
110, 14	*(nothing)*	with an adult
130, 14	alone	alone

Python matches an `else` with the `if` that starts in the same column above it. A and B differ only in the spaces before `else`, and they disagree: B tells a short teenager to ride with an adult and a tall child nothing. Line each `else` up with the test it answers.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints, or what happens; why, and the fix
`else if x < 0:`	`SyntaxError: expected ':'` `else if` is not Python: write `elif x < 0:`
`print(1)` `else:`	`IndentationError: unindent does not match any outer indentation level` this `else` is 2 spaces in, but its `if` is at 0 or 4: line them up
`# to do` `else:`	`IndentationError: expected an indented block after 'if' statement on line 1` a comment does not count as a block: write `pass`, a statement that does nothing
`print("checked")` `elif x < 0:`	`SyntaxError: invalid syntax` an `elif` must follow its `if` block directly: move `print("checked")` below the chain
`if count == True:`	`prints nothing` with `count = 3`: 3 is truthy, but `3 == True` is False. Write `if count > 0:`
`if answer:`	`prints: you said yes` with `answer = "False"`: every non-empty string is truthy. Compare the text: `if answer == "yes":`

12 Try it (answers in green)

1. A and B of the else-pairing panel, height 125, age 10?
A: `with an adult`. B: nothing.

2. Are 2100 and 2400 leap years?
False, True: 2100 fails the 400 test.

3. `bool(" ")`, `bool("")`, `bool(0.0)`?
`True False False`: a space is a character.

4. `not (x < 0 or x > 9)` without `not`?
`x >= 0 and x <= 9` (same for x from −2 to 11).

5. `if a:` holding `if b:` `print("both")`, as one `if`?
`if a and b:` (same in all four cases).

6. Moves 0, 1, 2: the number version with `u = 2`, `c = 0`?
`Comp wins: (2 + 1) % 3` is 0, which is `c`.

nested or chained, same tree · leap: 400, then 100, then 4 · 0, "", [] and None are falsy · else lines up with its if

Source: the floating point and conditionals notebook (its rock-paper-scissors versions and the hint to use numbers and %) and rps-1.py, nipunbatra.github.io/comp22. The ride, leap-year, truthiness and De Morgan examples are our own. Every output, error and path through a tree comes from running the code with Python 3.11.11. Next: the while loops sheet.